

Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question

06/02/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the `Lance` language with the ability to perform multiple assignments on the same line.

In the unmodified `Lance` language, assignments can only specify one variable on the left side of the `=` token, and one expression on the right-hand side of the `=` token. In addition to this syntax, comma-separated lists of variables and comma-separated lists of expressions shall be allowed to appear on the left and right sides of the `=` token respectively.

For an assignment to be syntactically correct, the length of the two lists must be the same. Additionally, the list on the left of the `=` token should not include array elements. When this is not the case, the compilation shall fail with a syntax error.

The order of variables in the list matches the order of expressions. In other words, the first expression is assigned to the first variable, the second expression is assigned to the second variable, and so on. The order in which the assignments are made in the compiled code is unspecified. Hence, dependencies between two expressions can be disregarded.

The following code snippet exemplifies the usage of the multiple assignment feature. In the snippet, three variables (`a`, `b` and `c`) are assigned by the same statement to three different values determined by the expressions on the right-hand side.

```
int a, b, c, d;

d = 10;
a, b, c = 5, 44 / 6, d * 2;

write(a); // 5
write(b); // 7
write(c); // 20
```

Hint: Be careful to not introduce ambiguities in the grammar. Indeed, the case in which the list of variables and expressions are of length one is already handled by the standard assignment operator grammar. Therefore, if in your grammar the lists can be of length one, *and no other modifications are made to the pre-existing rules*, the grammar will be ambiguous.

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..9a2356d 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -90,6 +90,12 @@ t_reg_allocator *RA;
+                               ↪ /* Register allocator. It implements the "Linear

t_io_infos *file_infos;      /* input and output files used by the compiler */

+/* The current list of variable identifiers to be assigned. Each element is
+ * of type char*. */
+t_list *assignVars = NULL;
+/* The current list of expressions to be assigned. Each element is of type
+ * t_axe_expression*. */
+t_list *assignExps = NULL;

extern int yylex(void);
extern void yyerror(const char*);
@@ -260,7 +266,103 @@ read_write_statement : read_statement { /* does nothing */ }
                        | write_statement { /* does nothing */ }

;

-assign_statement : IDENTIFIER LSQUARE exp RSQUARE ASSIGN exp
+assign_var_list: assign_var_list COMMA IDENTIFIER
+
+    {
+        /* Add the pointer to the variable identifier string to the
+         * end of the list of variables to be assigned. */
+        assignVars = addLast(assignVars, $3);
+    }
+    | IDENTIFIER
+    {
+        /* Same as the recursive case. */
+        assignVars = addLast(assignVars, $1);
+    }
+    ;
+
+assign_exp_list: assign_exp_list COMMA exp
+
+    {
+        /* Add the expression to the end of the list of expressions
+         * to be assigned.
+         * Notice that the type of $3 is a value type, as if
+         * $3 was a variable declared locally like this:
+         *     t_axe_expression $3;
+         * As a result, we cannot put the pointer to the expression
+         * $3 in the list because it is local to this semantic action
+         * and it would be deallocated at the end of it.
+         * To solve the issue we must dynamically allocate
+         * a new variable expression. */
+        t_axe_expression *expp = malloc(sizeof(t_axe_expression));
+        /* Copy the expression in the newly allocated buffer. */
+        *expp = $3;
+        /* Add the pointer to the buffer to the list. */
+        assignExps = addLast(assignExps, expp);
+    }
+    | exp
```



```

        * by a call to the function 'strdup' (see Acse.lex) */
        free($1);
    }
IDENTIFIER_ASSIGN_exp
{
    int location;

    /* in order to assign a value to a variable, we have to
    * know where the variable is located (i.e. in which register).
    * the function 'get_symbol_location' is used in order
    * to retrieve the register location assigned to
    * a given identifier.
    * A symbol table keeps track of the location of every
    * declared variable.
    * 'get_symbol_location' perform a query on the symbol table
    * in order to discover the correct location of
    * the variable with $1 as identifier */

    /* get the location of the symbol with the given ID. */
    location = get_symbol_location(program, $1, 0);

    /* update the value of location */
    if ($3.expression_type == IMMEDIATE)
        gen_move_immediate(program, location, $3.value);
    else
        gen_add_instruction(program,
                            location,
                            REG_0,
                            $3.value,
                            CG_DIRECT_ALL);

    /* free the memory associated with the IDENTIFIER */
    free($1);
}

;

if_statement : if_stmt
diff --git a/tests/multi_assign/multi_assign.src b/tests/multi_assign/multi_assign.src
new file mode 100644
index 00000000..46e782b
--- /dev/null
+++ b/tests/multi_assign/multi_assign.src
@@ -0,0 +1,8 @@
+int a, b, c, d;
+
+d = 10;
+a, b, c = 5, 44 / 6, d * 2;
+
+write(a); // 5
+write(b); // 7
+write(c); // 20

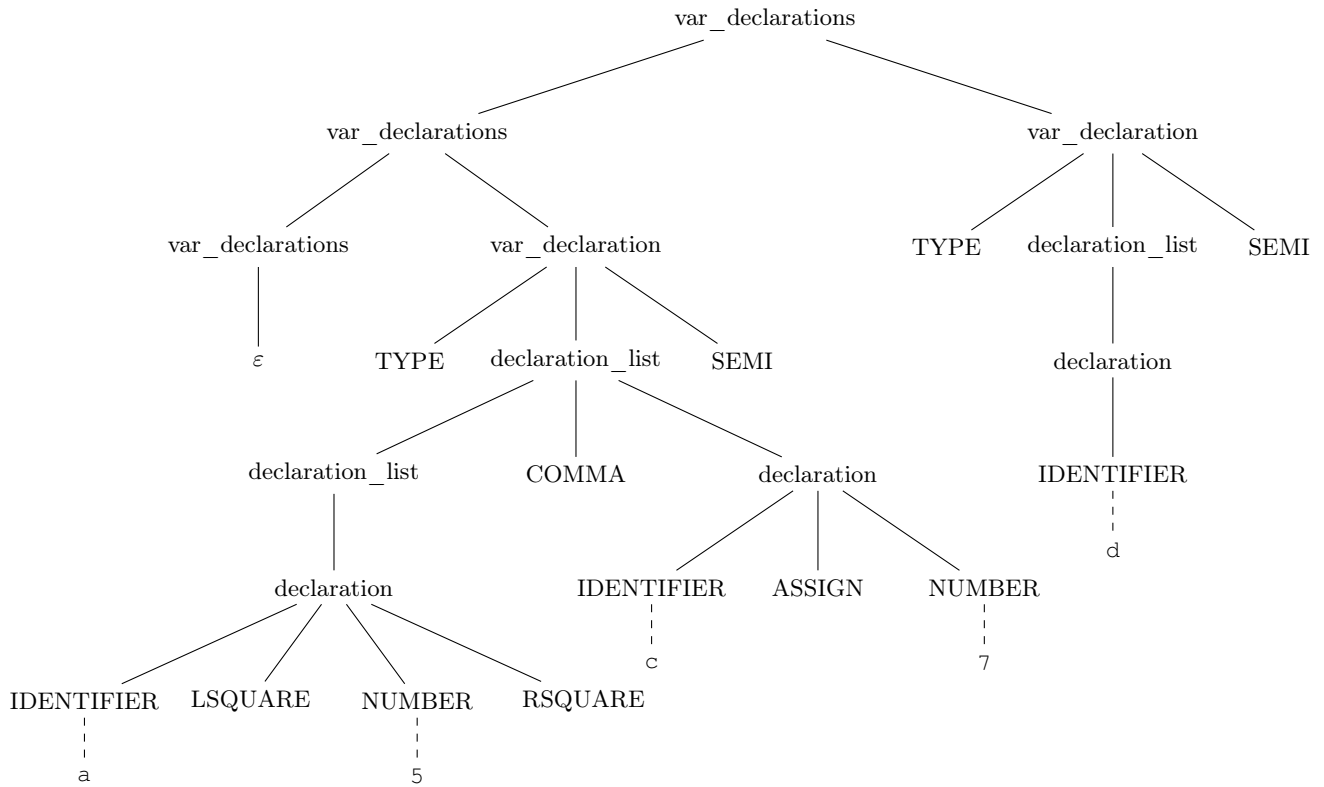
```

4. Given the following Lance code snippet:

```
int a[5], c=7;
int d;
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the var_declarations nonterminal*. (5 points)

Solution



5. (**Bonus**) Discuss how you would modify your solution to the previous exercises in order to add support for assignments to array elements within a multiple assignment statement.